

Autenticación y autorización con Spring Security

Qué es y qué problema resuelve

Es el mecanismo que responde dos preguntas en cada petición que llega a una API REST: ¿quién eres? (autenticación) y ¿qué puedes hacer? (autorización). Sin él, cualquier endpoint publicado queda abierto para cualquiera que sepa la URL.

La carpeta trae tres implementaciones progresivas del mismo problema, sobre la misma API de empleados, para que se vea qué cambia y qué se mantiene igual:

- **01-security-basic** — HTTP Basic: usuario y contraseña viajan (cifrados por HTTPS, en teoría) en cada petición, y Spring los valida contra la base de datos en cada una.
- **02-security-jwt** — el usuario se autentica una sola vez contra la base de datos y recibe un token firmado (JWT); de ahí en adelante manda ese token, no la contraseña.
- **03-security-oauth2** — la app deja de guardar usuarios y de emitir tokens: se convierte en un *resource server* puro que solo valida tokens emitidos por un proveedor externo (Keycloak).

Dónde se ve en el código

- **01-security-basic/.../security/SecurityConfig.java** — dos `@Bean`: `userDetailsService` (de dónde salen los usuarios: tablas `members/roles` vía `JdbcUserDetailsManager`) y `filterChain` (quién puede hacer qué, con `hasRole(...)` por método HTTP).
- **02-security-jwt/.../security/SecurityConfig.java** — ya no una cadena de filtros sino **dos**, con `@Order(1)` para `/api/auth/**` (login con HTTP Basic) y `@Order(2)` para el resto (solo acepta `Authorization: Bearer <token>`); además `jwtEncoder()/jwtDecoder()` con un par de llaves RSA.
- **02-security-jwt/.../rest/AuthController.java** — el único lugar de esa app donde todavía viaja la contraseña; construye el `JwtClaimsSet` y firma el token con la llave privada.
- **03-security-oauth2/.../security/SecurityConfig.java** — sin `UserDetailsService`, sin `JwtEncoder/Decoder` propios, sin `AuthController`: solo valida tokens de Keycloak y traduce sus claims (`realm_access.roles`) a los roles que Spring espera.
- **sql-scripts/01-security-tables.sql** — las tablas `members/roles` que usan los proyectos 01 y 02, con contraseñas guardadas como hash BCrypt (nunca en texto plano).

Las tres `SecurityConfig` comparten exactamente las mismas reglas de autorización (`GET` → `EMPLOYEE`, `POST/PUT/PATCH` → `MANAGER`, `DELETE` → `ADMIN`); lo único que cambia entre los tres proyectos es de dónde sale la identidad.

Qué pasa si no lo usas

- **Sin el @Bean filterChain (o sin Spring Security en el classpath):** todos los endpoints de EmployeeRestController quedan públicos. Cualquiera puede hacer DELETE /api/employees/1 sin credenciales.
- **Sin distinguir roles (hasRole) y usar solo authenticated():** cualquier usuario válido, aunque solo tenga permiso de lectura, podría borrar empleados — la autenticación por sí sola no implica autorización.
- **Escribiendo hasRole("ROLE_EMPLOYEE") en vez de hasRole("EMPLOYEE"):** Spring ya antepone el prefijo ROLE_ solo; con el prefijo escrito a mano busca ROLE_ROLE_EMPLOYEE, que no existe en ningún rol, y **todos** los usuarios reciben 403 aunque las credenciales sean correctas.
- **Sin sessionCreationPolicy(STATELESS) en una API sin formularios:** Spring crea una JSESSIONID en cada login, lo que agrega estado en el servidor que una API pensada para ser sin sesión no necesita, y complica escalar a varias instancias.
- **En el proyecto JWT, sin filtrar los claims que empiezan con FACTOR_ al construir el token:** el token saldría con roles internos de Spring (como FACTOR_PASSWORD) mezclados con los roles reales de negocio, filtrando detalle de implementación al cliente.

Cómo correrlo

```
docker run --name mysql-9.7 -p 3306:3306 -e MYSQL_ROOT_PASSWORD=tu_password -d
mysql:9.7
docker exec -i mysql-9.7 mysql -uroot -ptu_password employee_directory < sql-scripts/01-security-
tables.sql
```

Con la base lista, dentro de cada subcarpeta (01-security-basic, 02-security-jwt o 03-security-oauth2):

```
./mvnw spring-boot:run
./scripts/test-endpoints.sh # prueba la matriz de permisos con los 3 usuarios de práctic
```